Python

What Is Programming ?

Programming Is Like Giving Step-By-Step Instructions To A Computer To Do A Specific Task. Imagine You're Telling Someone How To Make A Sandwich:

1. **Get Two Slices Of Bread.**
2. **Spread Butter On One Side Of Each Slice.**
3. **Put A Slice Of Cheese On One Slice.**
4. **Put The Two Slices Together.**

In Programming, You Write Similar Instructions, But For A Computer. The Computer Doesn’t Understand Our Language, So We Use Special Languages (Like Python, Java, Or SQL) To Talk To It.

Just Like How You Follow A Recipe To Cook, A Computer Follows Your "Program" To Do Things Like Solve Math Problems, Play A Game, Or Display A Website.

In Other Word

Just Like We Use Hindi Or English To Communicate With Each Other ,We Use A Programming Language Like Python To Communicate With The Computer

What Is Python ?

Python Is A Popular Programming Language Known For Being Easy To Read And Write. It’s Great For Beginners Because The Code Looks Almost Like English, But It’s Also Powerful Enough For Big Projects. Python Is Used For A Wide Range Of Tasks, Such As:

- **Web Development**: Creating Websites.
- **Data Analysis**: Handling And Analyzing Large Amounts Of Data.
- **Machine Learning**: Teaching Computers To Learn From Data.
- **Automation**: Making Computers Do Repetitive Tasks Automatically.
- **Scripting**: Writing Small Programs To Perform Specific Tasks.

Features Of Python

Python Has Several Features That Make It Popular And Versatile For Many Types Of Projects. Here Are Some Key Features Of Python:

1. **Easy To Read And Write**: Python's Syntax Is Simple And Similar To English, Making It Easy For Beginners To Learn And Write Code.
2. **Cross-Platform Compatibility**: Python Works On Different Operating Systems Like Windows, Macos, And Linux, So You Can Write Code On One System And Run It On Another Without Modification.
3. **Large Standard Library**: Python Comes With A Rich Set Of Built-In Functions And Libraries For Tasks Like Web Development, File Handling, Data Manipulation, And More. This Reduces The Need To Write Code From Scratch.
4. **Open Source And Free**: Python Is Open-Source, Meaning Anyone Can Use, Modify, And Distribute It For Free.
5. **Dynamically Typed**: You Don’t Need To Declare The Type Of A Variable (Like Integer, String, Etc.) Before Using It. Python Figures It Out During Execution.
6. **Object-Oriented**: Python Supports Object-Oriented Programming (Which Focuses On Objects And Classes)
7. **Automation-Friendly**: Python Is Often Used For Automating Repetitive Tasks, Like Moving Files, Renaming Documents, Or Scraping Websites For Data.

Because Of These Features, Python Is Used In A Wide Variety Of Fields, Including Web Development, Data Science, Machine Learning, Automation, And More.

Python's history :

1. **Who made it?**
    - **Guido van Rossum** created Python in 1989 as a fun project during Christmas.
2. **First version**
    - The first version, **Python 1.0**, was released in 1991.
3. **Why is it called Python?**

- Guido named it after a funny TV show called **"Monty Python's Flying Circus"**, not the snake.

Installation

Here’s A Step-By-Step Guide To Installing Python On Your Computer.

1. Download Python

- Go To The Official Python Website: Python.Org.
- You'll See A Button That Says **Download Python X.X.X** (The Latest Version).
- Click That Button To Download The Installer.

2. Run The Installer

- Open The Downloaded File To Run The Installer.
- **Important:** On The First Screen, Check The Box That Says **Add Python To PATH**. This Ensures Python Can Be Run From The Command Line.
- Then Click **Install Now**.

3. Verify The Installation

- Once The Installation Is Complete, Open A Terminal Or Command Prompt.
  - On Windows, Search For "Command Prompt."
  - On Macos/Linux, Open "Terminal."
- Type The Following Command To Check If Python Is Installed And Working:

Python --Version

You Should See Something Like Python 3.X.X, Which Confirms Python Is Installed.

4. Install Pip (Python’s Package Manager)

- **Pip** Is A Tool That Helps You Install Python Libraries. It Usually Comes With Python By Default.

- To Verify Pip Is Installed, Type:

Pip --Version

If It's Installed, You'll See A Version Number.

5. Install A Text Editor/IDE

You’ll Need A Place To Write And Run Your Python Code. Some Popular Options Are:

- **VS Code** (Visual Studio Code): A Lightweight, Feature-Rich Editor.
- **Pycharm**: A More Powerful IDE Designed Specifically For Python Development.
- **Jupyter Notebook**: Great For Data Analysis And Scientific Computing.

6. Running Python Code

- To Run A Python File, Save It With A .Py Extension (E.G., Hello.Py).
- Install Extentsion The Name Of Extension Is Python

Chapter 1

(First Code & Comments)

Let’s Write Our Very First Python Programe Create A File Called Hello.Py And Paste The Bellow Code In It

Print(“Hello World”)

(Print Is A Function That Print The Any Value)

Run This File (Hello.Py) And You Will See Hello World Printed On You Screen

To print the multiline of using single print functon use (’’’xyz’’’) triple inverted comma

Example-print(’’’hello

World’’’)

Modules

A Module Is A File Containing Code Written By Somebody

Else Which Can Be Imported And Used In Our Programs

Pip

Pip Is The Package Manager For Python You Can Use Pip Install A Modules On Your System

Example-Pip Install Pandas

Type Of Modules

Built Is Modules -Pre Install In Python

External Modules – Need To Install Using Pip

Some Example Of Built In Modules Are Os ,Abc Etc.

Some Example Of External Modules Are Tensorflow,Flask Etc.

Python As Calculator

We Can Use Python As A Calculator In Two Ways

First

Use Directly In Python File Like

Print(10+10)

Print(10-10) Etc.

Second

Using Terminal

Without Use Print Function

10+10

10-10 Etc.

Comments

Comments Are Used To Explain Code And Make It More Readable. They Are Ignored By The Python Interpreter During Execution. Comments Are Particularly Helpful For Documenting Your Code For Others (Or Yourself) When You Return To It Later. There Are Two Types Of Comments In Python: **Single-Line Comments** And **Multi-Line Comments**.

In Ather Word

Comments Are Used To Write Something Which The Programmer Does Not Want To Execute.

1. Single-Line Comments:

- A Single-Line Comment Starts With A # Symbol And Extends To The End Of The Line.
- Everything After The # On That Line Is Ignored By Python.

2. Multi-Line Comments:

Python Doesn’t Have A Specific Syntax For Multi-Line Comments Like Other Languages (Such As /* ... */ In C Or Java). However, You Can Create Multi-Line Comments Using Multiple # Symbols, Or By Using Triple Quotes (''' Or """,’’’Or’’’) For Block Comments Or Docstrings.

Practice Questions

1. Write a programe to print a poem in python.
2. Write a program to fill you details like this:
   (name = sumit
   Age = 22
   City = sikar)
3. Write a programe to print your name only.
4. Write a program to comments the Q2 using multiline comments.
5. Write a program to comment Q3 using single line comment.
6. Write a program to sum two number.
7. Write a program to find the average of 3 number.
8. Write a program and try to sum two values using terminal
9. Write a program and print name and age using single print function.
10. Write a program and print the multiplication table of 2.

Chapter 2

Variable and datatypes

DATA TYPE

There are two type of data type in python .

1.primarily data type

2.non.primarily data type.

Primarily there are following data types in python

1 integer	(1,2,3,54,4,25,10)
2 floating point number	(10.5,20.5,10.6,5.2)
3 string	(“hello world”,”rohan”)
4 booleans	(True,False)
5 none	(nonetype)

variables are used to store data values. Here are the key points about variables in Python:

1. Creating Variables

- Variables are created when you assign a value to them.
- You don’t need to declare the type of a variable explicitly; Python automatically infers the type based on the value assigned.

Examples-

```
x = 5  # integer

y = "Hello"  # string

z = 3.14  # float
```

2. Variable Naming Rules

- Must start with a letter (a-z, A-Z) or an underscore (_).
- The rest of the name can include letters, digits (0-9), and underscores.
- Variable names are case-sensitive (age, Age, and AGE are different variables).
- No while space is allowed to be used inside a variable name.
- A variable name can’t start with the digit.

Example-

```
my_var = 10

MyVar = 15

_my_var = 20
```

3. Types of Variables

Python automatically assigns the type to the variable based on the value. You can check the type using the type() function.

```
a = 42

b = "Python"



print(type(a))  # <class 'int'>

print(type(b))  # <class 'str'>
```

4. Dynamic Typing

You can reassign variables to different data types in Python. For example:

Examples-

x = 10 # Initially an integer

x = "Ten" # Now a string

operators in python

operators are special symbols that perform operations on one or more operands. Here's a summary of the main types of operators available in Python:

1. Arithmetic Operators

These operators perform basic mathematical operations.

Operator	Description	Example
+	Addition	5 + 3 → 8
-	Subtraction	5 - 3 → 2
*	Multiplication	5 * 3 → 15
/	Division	5 / 2 → 2.5
//	Floor Division	5 // 2 → 2
%	Modulus (Remainder)	5 % 2 → 1
**	Exponentiation	2 ** 3 → 8

2. Comparison Operators

These operators compare two values and return a boolean result.

Operator	Description	Example
==	Equal to	5 == 3 → False
!=	Not equal to	5 != 3 → True
>	Greater than	5 > 3 → True

Operator	Description	Example
<	Less than	5 < 3 → False
>=	Greater than or equal to	5 >= 3 → True
<=	Less than or equal to	5 <= 3 → False

3. Logical Operators

These operators are used to combine conditional statements.

Operator	Description	Example
and	Logical AND	True and False → False
or	Logical OR	True or False → True
not	Logical NOT	not True → False

4. Assignment Operators

These operators are used to assign values to variables.

Operator	Description	Example
=	Assign	x = 5
+=	Add and assign	x += 3 → x = x + 3
-=	Subtract and assign	x -= 3 → x = x - 3
*=	Multiply and assign	x *= 3 → x = x * 3
/=	Divide and assign	x /= 3 → x = x / 3

Casting

A number can be converted into a string and athor data type

Examples-

Str(31) =”31”

Int(“32”)=32

Taking input from user

Input() function

This function allowe the user to take input from the keyboard as a string.

Name = input(“enter your name”)

It is inported to note that the output of input is always a string even if the number is enter

If you want to input inter then use casting(int) to change in to int.

Example-

Age = int(input(“enter your age “)

Practice Questions

1.write a python program to find the average of 5 number take number from user.

2.write a python program to sum 3 value the values take from user .

3.check the type of the veriable assigned using input() function.

4.write a code to find the remainder when a number in devided by 2.

5.compare the value num1 and num2 and find out whether a num1 is greter then num2 or not

Take num1 = 50,num2 =20

6.write a python program to calculate square of a number enterd by the user

7.find the power or a number enter by the user.

8.write a program that great a user enter name followed by hello .

9.write a program to take 2 input from user first name and last name and show in single line.

10.write a program and take input from user and conver into int and add 5 number .

Chapter 3.

String

In Python, a string is a sequence of characters enclosed in either single quotes ('), double quotes ("), or triple quotes (''' or """). Here are some basic concepts and operations you can perform with strings:

Creating Strings

```
# Single-quoted string

single_quote_string = 'Hello, World!'


# Double-quoted string

double_quote_string = "Hello, World!"


# Triple-quoted string (can span multiple lines)

triple_quote_string = '''Hello,

World!'''
```

String Operations

Concatenation: You can join two or more strings using the + operator.

```
str1 = "Hello"

str2 = "World"

result = str1 + " " + str2  # "Hello World"
```

Repetition: You can repeat a string multiple times using the * operator.

```
repeated = "Hi! " * 3  # "Hi! Hi! Hi! "
```

Accessing Characters: You can access individual characters using indexing (starting from 0).

```
my_string = "Python"

first_char = my_string[0]  # 'P'

last_char = my_string[-1]   # 'n'
```

Slicing: You can get a substring by slicing.

```
substring = my_string[1:4]  # 'yth' (characters from index 1 to 3)
```

String Length: Use the len() function to find the length of a string.

```
length = len(my_string)  # 6
```

String Methods

Python provides various built-in methods to manipulate strings:

str.lower(): Converts the string to lowercase.

```
my_string.lower()  # 'python'
```

str.upper(): Converts the string to uppercase.

```
my_string.upper()  # 'PYTHON'
```

str.strip(): Removes whitespace from the beginning and end of the string.

```
my_string.strip()  # Removes leading and trailing whitespace
```

str.split(separator): Splits the string into a list of substrings.

words = "Hello, World!".split(", ") # ['Hello', 'World!']

str.replace(old, new): Replaces a substring with another substring.

new_string = my_string.replace("Py", "Ja") # 'Jathon'

str.endswith(“sh”) whether a text endswith sh or not

new_string = my_string.endswith(“on”)   #True

str.count(“a”) count the total number of any character

str.capitalize() convert to capitalize

str.find(“word”) this function find the index number of given word

str.startswith(“py”) wether a word start with py or not.

Escape seqences characters

Escape sequences in Python are special characters that allow you to represent certain characters that may be difficult to include directly in a string. They start with a backslash (\) followed by a character. Here's a list of commonly used escape sequences:

Common Escape Sequences

Escape Sequence	Description
\'	Single quote
\"	Double quote
\\	Backslash
\n	New line (line break)
\t	Tab

Practice Questions

1.Write a Python program that concatenates two strings, "Hello" and "World", with a space in between and prints the result.

2.Create a program that takes a string input from the user and prints the length of that string.

3.Given the string s = "Python Programming", extract and print the substring "Programming" using slicing

4. Write a program that replaces all occurrences of the word "bad" with "good" in the string s = "This is a bad example of a bad string." and prints the new string.

5.Create a program that takes a user's name and age as input, then formats and prints the string in the following format: "My name is [name] and I am [age] years old."

6.Create a function that takes a string as input and returns the reversed string. For example, if the input is "Python", the output should be "nohtyP".

7. Write a program that prints the following string using escape sequences:
She said, "Its a beautiful day!"

8. Given the string s = "apple,banana,cherry", split the string into a list of fruits and then join them back into a single string separated by a space.

9.Write a program to display a user enterd name followed by good afternoon followed by good morning using input()function.

10.write a program to dedect dubble space in a string

11.replace double space from problem 10 with single space

12.write a program to fill in a letter templete given bellow wih name and date.

Letter = ‘’’dear <name>,

You are selected!

<date>

Chapter 4

(List)

list is a collection of items that are ordered, changeable (mutable), and allow duplicate values. Lists are one of the most commonly used data structures in Python. You can create lists, modify them, and perform a variety of operations on them.

Creating a List

You can create a list by placing items inside square brackets [], separated by commas:

```
# Creating a list of numbers

numbers = [1, 2, 3, 4, 5]


# Creating a list of strings

fruits = ['apple', 'banana', 'cherry']


# Creating a mixed data type list

mixed_list = [1, 'hello', 3.14, True]
```

Accessing List Items

You can access items in a list using their index (starting from 0):

```
# Accessing the first item

print(numbers[0])  # Output: 1
```

```
# Accessing the second item in the fruits list
print(fruits[1])  # Output: banana
```

Modifying a List

Lists are mutable, so you can change their items:

```
fruits[1] = 'blueberry'
print(fruits)  # Output: ['apple', 'blueberry', 'cherry']
```

Adding Items to a List

You can add items to a list using the append() method, insert() method, or by using the + operator:

```
# Using append() to add an item to the end of the list
fruits.append('orange')
print(fruits)  # Output: ['apple', 'blueberry', 'cherry', 'orange']

# Using insert() to add an item at a specific index
fruits.insert(1, 'banana')
print(fruits)  # Output: ['apple', 'banana', 'blueberry', 'cherry', 'orange']
```

Removing Items from a List

You can remove items from a list using methods like remove(), pop(), or del:

```
# Removing a specific item by value
fruits.remove('banana')
print(fruits)  # Output: ['apple', 'blueberry', 'cherry', 'orange']

# Removing an item by index using pop()
fruits.pop(2)
print(fruits)  # Output: ['apple', 'blueberry', 'orange']

# Deleting an item by index using del
del fruits[1]
print(fruits)  # Output: ['apple', 'orange']
```

List Methods

Here are some common list methods:

- append(): Adds an item to the end of the list.
- insert(index, item): Inserts an item at a specific position.
- remove(item): Removes the first occurrence of the specified item.
- pop(index): Removes and returns the item at the specified position.

- sort(): Sorts the list in ascending order.
- reverse(): Reverses the order of the list.
- len(): Returns the number of items in the list.

Here are some additional methods and operations you can use with lists in Python, along with a brief explanation of each:

List Methods

1. **extend(iterable)**: Adds all the elements of an iterable (like another list) to the end of the current list.

```
numbers = [1, 2, 3]

numbers.extend([4, 5, 6])

print(numbers)  # Output: [1, 2, 3, 4, 5, 6]
```

2. **index(element)**: Returns the index of the first occurrence of the specified element.

```
fruits = ['apple', 'banana', 'cherry']

print(fruits.index('banana'))  # Output: 1
```

3. **count(element)**: Returns the number of times the specified element appears in the list.

```
numbers = [1, 2, 3, 2, 4, 2]

print(numbers.count(2))  # Output: 3
```

4. **clear()**: Removes all elements from the list, making it empty.

```
numbers = [1, 2, 3, 4]
numbers.clear()
print(numbers)  # Output: []
```

5. **copy()**: Returns a shallow copy of the list.

```
original_list = [1, 2, 3]
copied_list = original_list.copy()
print(copied_list)  # Output: [1, 2, 3]
```

Built-in Functions with Lists

These functions are often used with lists:

1. **len(list)**: Returns the number of elements in the list.

```
numbers = [1, 2, 3, 4, 5]
print(len(numbers))  # Output: 5
```

2. **max(list)**: Returns the largest element in the list.

```
numbers = [10, 20, 30, 40]
print(max(numbers))  # Output: 40
```

3. **min(list)**: Returns the smallest element in the list.

```
numbers = [10, 20, 30, 40]
print(min(numbers))  # Output: 10
```

4. **sum(list)**: Returns the sum of all elements in the list (elements must be numeric).

```
numbers = [1, 2, 3, 4, 5]

print(sum(numbers))  # Output: 15
```

5. **sorted(list)**: Returns a new list containing all elements from the original list in sorted order.

```
numbers = [3, 1, 4, 2]

sorted_numbers = sorted(numbers)

print(sorted_numbers)  # Output: [1, 2, 3, 4]
```

Additional Operations

1. **List Concatenation**: You can concatenate two lists using the + operator.

```
list1 = [1, 2, 3]

list2 = [4, 5, 6]

combined = list1 + list2

print(combined)  # Output: [1, 2, 3, 4, 5, 6]
```

2. **List Repetition**: You can repeat the elements in a list using the * operator.

```
numbers = [1, 2, 3]

repeated = numbers * 2

print(repeated)  # Output: [1, 2, 3, 1, 2, 3]
```

3. **Check if an Element Exists**: Use the in operator to check if an element is in the list.

```
fruits = ['apple', 'banana', 'cherry']

print('banana' in fruits)  # Output: True

print('orange' in fruits)  # Output: False
```

Practice

1. Write a program to find the largest number in a list.

2. Write a program to find the length of a list.

3. Given a list of numbers, find and print the sum of all numbers in the list.

4. Write a program that asks the user to enter 5 numbers and stores them in a list. Print the list in Accending order.

5. Ask the user to input their three favorite fruits and store them in a list. Print a message saying "Your favorite fruits are: " followed by the list of fruits.

6. Create a list of items and ask the user to enter an item. Check if the item is in the list and print a message indicating whether it is found or not.

7. Ask the user to enter numbers until they enter a negative number. Store the numbers in a list and print the length of the list.

8. Ask the user to enter 10 numbers, one by one. Store them in a list and then calculate and print the sum of all numbers in the list.

9. Ask the user to enter a list of 5 items. Then, ask them to enter one item to remove. Remove that item from the list if it exists and display the updated list.

10. Write a program that takes a list of numbers from the user and then asks the user to enter a number. Print the index of the number in the list if it exists.

Chapter 5

(tuples)

A tuple is a collection in Python that is similar to a list but with a key difference: tuples are immutable, meaning they cannot be changed once they are created. Tuples are useful for storing data that should not be modified. They are defined using parentheses () instead of square brackets [].

Creating a Tuple

You can create a tuple by placing elements inside parentheses, separated by commas:

```
# Creating a tuple of numbers

numbers = (1, 2, 3, 4, 5)


# Creating a tuple of strings

fruits = ('apple', 'banana', 'cherry')


# Creating a mixed data type tuple

mixed_tuple = (1, 'hello', 3.14, True)


# Creating a tuple with a single element (note the comma)

single_element_tuple = (5,)
```

Accessing Tuple Elements

You can access elements of a tuple using their index, just like with lists:

```
# Accessing the first element

print(fruits[0])  # Output: apple


# Accessing the last element

print(fruits[-1])  # Output: cherry
```

Tuple Immutability

Tuples are immutable, meaning you cannot change, add, or remove items after the tuple is created:

```
# Attempting to change a tuple element will raise an error

fruits[1] = 'orange'  # TypeError: 'tuple' object does not support item assignment
```

Tuple Operations

Although you can't modify a tuple, there are some operations you can perform with tuples:

Concatenation: Combine two tuples using the + operator.

```
tuple1 = (1, 2, 3)

tuple2 = (4, 5, 6)

combined_tuple = tuple1 + tuple2
```

```
print(combined_tuple)  # Output: (1, 2, 3, 4, 5, 6)
```

Repetition: Repeat the elements in a tuple using the * operator.

```
fruits = ('apple', 'banana')

repeated_fruits = fruits * 2

print(repeated_fruits)  # Output: ('apple', 'banana', 'apple', 'banana')
```

Membership Test: Check if an element exists in a tuple using the in operator.

```
print('banana' in fruits)  # Output: True

print('orange' in fruits)  # Output: False
```

Slicing: You can slice tuples to get a portion of them.

```
numbers = (1, 2, 3, 4, 5, 6)

sliced_tuple = numbers[1:4]

print(sliced_tuple)  # Output: (2, 3, 4)
```

Tuple Methods

Tuples have a limited number of methods compared to lists, as they are immutable. The most common tuple methods are:

count(): Returns the number of times a specified element appears in the tuple.

```
numbers = (1, 2, 2, 3, 4, 2)

print(numbers.count(2))  # Output: 3
```

index(): Returns the index of the first occurrence of a specified element.

```
fruits = ('apple', 'banana', 'cherry')

print(fruits.index('banana'))  # Output: 1
```

When to Use Tuples

Data Integrity: Use tuples when you want to ensure that the data should not be modified.

Performance: Tuples can be slightly faster than lists due to their immutability, making them more efficient when working with large datasets.

Dictionary Keys: Tuples can be used as keys in dictionaries, while lists cannot because they are mutable.

Example

Here is an example of creating and using a tuple:

```
# Defining a tuple with student data

student = ('John', 21, 'Physics')


# Accessing tuple elements

print(f"Name: {student[0]}")      # Output: Name: John

print(f"Age: {student[1]}")       # Output: Age: 21

print(f"Major: {student[2]}")     # Output: Major: Physics
```

Practice

1.Given the tuple my_tuple = (1, 2, 3, 4, 5), print the second element.

2. Use slicing to get the last three elements from the tuple numbers = (5, 10, 15, 20, 25, 30).

3. Write a program to find the length of the tuple fruits = ('apple', 'banana', 'cherry', 'date').

4. Try to change the second element of the tuple my_tuple = (1, 2, 3) to 5. What happens? Explain why.

5. Given two tuples, tuple1 = (1, 2, 3) and tuple2 = (4, 5, 6), write a program to concatenate them into a new tuple.

6. Write a Python function that takes a tuple and an element as input and checks if the element exists in the tuple.

7. Write a program to find the maximum and minimum values in the tuple numbers = (2, 8, 1, 5, 10).

8. Given the tuple animals = ('cat', 'dog', 'rabbit', 'hamster'), find the index of the element 'rabbit'.

9. Write a program to count how many times the number 3 appears in the tuple numbers = (1, 3, 5, 3, 7, 9, 3).

10.Given the tuple values = (10, 25, 3, 40, 15), write a program to find the largest element in the tuple.

(chapter 6)

Dictionary & Sets

Dictionary

A dictionary in Python is a collection of key-value pairs. Each key must be unique, and it maps to a specific value. Dictionaries are useful when you need to store data that can be accessed using a unique identifier (the key).

Key Features of a Dictionary:

Unordered: The items in a dictionary are not stored in any specific order.

Mutable: You can change, add, or remove items after the dictionary is created.

Unique Keys: Each key in the dictionary must be unique.

Key-Value Pairs: Each entry in a dictionary consists of a key and its associated value.

Creating a Dictionary

You can create a dictionary using curly braces {} or the dict() function.

```
# Using curly braces

student = {'name': 'John', 'age': 21, 'major': 'Computer Science'}


# Using the dict() function

employee = dict(name='Alice', age=30, position='Manager')
```

Accessing Dictionary Elements

You can access values in a dictionary using their keys.

```
print(student['name'])  # Output: John

print(employee['position'])  # Output: Manager
```

Modifying a Dictionary

You can add new key-value pairs or update existing ones.

```
# Adding a new key-value pair

student['grade'] = 'A'


# Updating an existing key-value pair

student['age'] = 22

# Output: {'name': 'John', 'age': 22, 'major': 'Computer Science', 'grade':
'A'}

print(student)
```

Removing Items from a Dictionary

You can remove items using the del statement or the pop() method.

```
# Using del to remove a key-value pair

del student['major']
```

```
# Using pop() to remove a key-value pair and get its value
age = student.pop('age')


# Output: {'name': 'John', 'grade': 'A'}
print(student)
```

Dictionary Methods

Here are some common dictionary methods:

keys() – Returns a view of the dictionary's keys.

values() – Returns a view of the dictionary's values.

items() – Returns a view of the dictionary's key-value pairs.

get(key) – Returns the value for a key if the key is in the dictionary.

pop(key) – Removes the item with the specified key and returns its value.

Set

A set in Python is an unordered collection of unique elements. Sets are commonly used when you need to store non-duplicate items, and you don't care about the order of the elements.

Key Features of a Set:

Unordered: Sets do not maintain the order of elements.

Unique Elements: Sets automatically remove duplicate items.

Mutable: You can add or remove items from a set.

No Indexing: Since sets are unordered, they do not support indexing or slicing.

Creating a Set

You can create a set using curly braces {} or the set() function.

```
# Creating a set with curly braces
my_set = {1, 2, 3, 4, 5}

# Creating a set using the set() function
another_set = set([1, 2, 3, 4, 5, 5, 6])  # Duplicate 5 will be removed

# Output: {1, 2, 3, 4, 5, 6}
print(another_set)
```

Adding Elements to a Set

You can add items to a set using the add() method.

```
my_set.add(6)  # Now my_set is {1, 2, 3, 4, 5, 6}
```

Removing Elements from a Set

You can remove items using the remove() or discard() method. The difference between these two is that remove() raises an error if the item is not found, while discard() does not.

```
# Using remove()

my_set.remove(3)  # Removes the element 3


# Using discard()

my_set.discard(10)  # Does nothing since 10 is not in the set


set1 = {1, 2, 3, 4}

set2 = {3, 4, 5, 6}


# Union

print(set1 | set2)  # Output: {1, 2, 3, 4, 5, 6}


# Intersection

print(set1 & set2)  # Output: {3, 4}


# Difference

print(set1 - set2)  # Output: {1, 2}
```

Set Methods

Some common set methods include:

add(element) – Adds an element to the set.

remove(element) – Removes the specified element (raises an error if the element is not found).

discard(element) – Removes the specified element (does not raise an error if the element is not found).

pop() – Removes and returns an arbitrary element from the set.

clear() – Removes all elements from the set.

```
# Example of using set methods
my_set = {1, 2, 3}
my_set.add(4)        # Adds 4 to the set
my_set.remove(2)     # Removes 2 from the set
element = my_set.pop() # Removes and returns an arbitrary element
my_set.clear()
```

Practice

1.Create a dictionary called student_info with the following data:

- Name: "John"
- Age: 22
- Major: "Physics"

Add the key-value pair 'grade': 'A' to the student_info dictionary. Then, update the Age to 23.

2.write a program to create a dictionary of hindi word with values as their English translation provide user with an option to look it up.

3.can we have a set with 15(int) and “18”(str) as a values in it?.

4. what will be the length of following set s

S = set()

s.add(20)

s.add(20.0)

s.add(“20”)

5.n ={}

What is the type of s ?

6.create a empty dictionary allow 4 friends to enter their fav.language as values and use keys as their name assume that the name are unique

7.if name of 2 friends are same what will to thee program in problem 6.

8.if language of two friends are same what will happen to the program in problem 6.

9.write a program and take a input from the user and save as unique values.

10.write a program to input eight number from the user and display all the unique name.

Chapter 7

(Conditional Expressions)

Conditional expressions in Python allow you to make decisions in your code based on conditions. The most common way to write conditional logic in Python is with if, elif, and else statements. Here's a simple explanation with examples.

Basic Syntax

```
if condition:
    # Code to execute if the condition is True
elif another_condition:
    # Code to execute if the second condition is True
else:
    # Code to execute if none of the conditions are True
```

Example 1: Simple if Statement

```
age = 18


if age >= 18:
    print("You are an adult!")
```

This code checks if the value of age is greater than or equal to 18.

If the condition is True, it prints "You are an adult!"

Example 2: Using if, elif, and else

```
age = 15


if age >= 18:
    print("You are an adult!")
elif age >= 13:
    print("You are a teenager!")
else:
    print("You are a child!")
```

if: Checks the first condition. If it's True, it runs the code inside it and skips the rest.

elif: Stands for "else if." It checks another condition if the first one is False.

else: Runs if none of the previous conditions are True.

In this example:

If age is 18 or more, it prints "You are an adult!"

If age is between 13 and 17, it prints "You are a teenager!"

Otherwise, it prints "You are a child!"

Example 3: Conditional (Ternary) Operator

Python also has a shorthand way to write conditional expressions using the ternary operator. It allows you to write simple if-else statements in a single line.

```
age = 20

message = "Adult" if age >= 18 else "Not an adult"

print(message)
```

This one-liner checks if age is 18 or more.

If True, it sets message to "Adult"; otherwise, it sets message to "Not an adult."

Key Points

Indentation: Python uses indentation (spaces or tabs) to define the code blocks for each condition. This means you must indent the code correctly for it to work.

Comparison Operators: Common operators include == (equal to), != (not equal to), >, <, >=, and <=.

Logical Operators: You can use and, or, and not to combine multiple conditions.

Practice

1.Write a Python program that checks if a given number is positive, negative, or zero and prints the appropriate message.

2. Write a program that takes two numbers as input and prints which one is larger using a conditional expression.

3. Write a program that checks if a person is eligible to vote. The minimum voting age is 18.

4. Write a program that calculates a grade based on the following conditions:

- Score ≥ 90: Grade A
- 80 ≤ Score < 90: Grade B
- 70 ≤ Score < 80: Grade C
- 60 ≤ Score < 70: Grade D
- Score < 60: Grade F

5.Write a program that checks if a given number is a prime number and prints "Prime" or "Not Prime."

6. Write a program that calculates simple interest. If the interest is greater than $100, print "High interest"; otherwise, print "Low interest.".

7. Write a program that prints what action to take based on the color of a traffic light:

- "Stop" for red
- "Caution" for yellow

- "Go" for green

8.Write a program that takes two numbers and an operator (+, -, *, /) and performs the operation, printing the result. Handle division by zero by printing "Error: Division by zero."

9.write a program to find whether a given username cantains less then 10

Character or not.

10.write a program to find greatest of faur number entered by the user.

Chapter 8

(loops)

Loops in Python allow you to repeat a block of code multiple times, which makes your code more efficient and easier to write. There are two main types of loops in Python: for loops and while loops. Let's go through each one in detail.

1. for Loop

A for loop is used when you need to iterate over a sequence (like a list, tuple, string, or range) and know the number of times you want to execute a block of code.

Syntax:

```
for item in sequence:
    # Code to repeat
```

Example: Printing numbers from 1 to 5 using a for loop.

```
for i in range(1, 6):
    print(i)
```

- range(1, 6) creates a sequence of numbers starting from 1 up to (but not including) 6.
- The loop runs once for each number in the sequence, printing the value of i.

Output:

```
1
```

2

3

4

5

Using for Loop with Lists

You can also use a for loop to iterate through each item in a list.

Example:

```
fruits = ['apple', 'banana', 'cherry']

for fruit in fruits:

    print(fruit)
```

Output:

apple

banana

cherry

2. while Loop

A while loop runs as long as a condition is True. It’s useful when you don’t know how many times you need to repeat the code.

Syntax:

```
while condition:

    # Code to repeat
```

Example: Printing numbers from 1 to 5 using a while loop.

python

Copy code

```
i = 1

while i <= 5:

    print(i)

    i += 1  # Increase i by 1 after each loop
```

- The loop continues to run as long as i is less than or equal to 5.
- After each iteration, i is increased by 1 using i += 1.

Output:

1

2

3

4

5

Infinite Loops

An infinite loop is a loop that runs forever if the stopping condition is never met. To avoid infinite loops, make sure the loop condition will eventually become False.

Example: Infinite loop (avoid doing this without a break condition):

```
while True:

    print("This will run forever unless you stop it!")
```

Breaking Out of Loops

You can use the break statement to stop a loop before it completes all its iterations.

Example:

```
for i in range(1, 10):
    if i == 5:
        break  # Stop the loop when i equals 5
    print(i)
```

Output:

```
1
2
3
4
```

Skipping Iterations

The continue statement lets you skip the rest of the code in the current iteration and move to the next iteration.

Example:

```
for i in range(1, 6):
    if i == 3:
        continue  # Skip the rest of the code when i equals 3
    print(i)
```

Output:

1

2

4

5

Pass statement

The pass statement in Python is used when you want to write a part of your code but don’t want to do anything yet. It acts as a placeholder.

```
for i in range(3):
    pass                # Nothing happen s in the loop
```

Practice

1. Print Numbers: Write a program that prints the numbers from 1 to 10.
2. Sum of Numbers: Write a program that calculates the sum of all numbers from 1 to 100.
3. Even Numbers: Write a program that prints all even numbers between 1 and 50.
4. Odd Numbers: Write a program that prints all odd numbers between 1 and 50.
5. Multiplication Table: Write a program that prints the multiplication table of 5 (from 1 to 10).
6. Factorial Calculation: Write a program that calculates the factorial of a given number using a loop.
7. Countdown: Write a program that counts down from 10 to 1 and then prints "Blast off!".
8. List of Squares: Write a program that prints the squares of numbers from 1 to 10.
9. String Reversal: Write a program that takes a string and prints it in reverse order using a loop.
10. Write a program a program to great all the person name starts in a list and which start with ‘s’.
    L1=[‘vikash’,’sohan’,’sachin’,’suresh’]

Chapter 9
(function)

What is a Function in Python?

A function in Python is a block of code that performs a specific task. You can think of it as a small program that you can use whenever you want without having to write the same code again. Functions help make your code organized, reusable, and easier to read.

Why Use Functions?

- Reusability: Write code once and use it multiple times.
- Organization: Break down complex problems into smaller, manageable pieces.
- Clarity: Make your code more understandable by naming functions according to their tasks.

How to Define a Function

To create a function in Python, you use the def keyword, followed by the function name and parentheses. Here's the basic syntax:

```
def function_name(parameters):
    # Code block to be executed
    return result
```

- function_name: The name you give to the function (e.g., add_numbers).
- parameters: Optional inputs you can pass into the function (e.g., x and y).

- return: The value that the function gives back after executing.

Example 1: A Simple Function

Here’s a simple function that adds two numbers:

```
def add_numbers(x, y):
    return x + y
```

- Definition: This function is named add_numbers and takes two parameters, x and y.
- Action: It adds the two numbers and returns the result.

Using the Function

To use (or call) the function, you simply write its name and pass in the required arguments:

```
result = add_numbers(3, 5)
print(result)  # Output: 8
```

Example 2: A Function Without Parameters

You can also create functions that don’t take any parameters. Here’s an

```
def greet():
    print("Hello, welcome to Python!")
```

- Definition: This function is called greet and doesn’t take any inputs.
- Action: It prints a greeting message when called.

Using the Function

You can call the function like this:

```
greet()  # Output: Hello, welcome to Python!
```

Example 3: Function with Default Parameters

You can give default values to parameters. If the caller does not provide a value, the default is used:

```
def multiply(a, b=2):
    return a * b
```

- Definition: This function multiplies a by b, where b defaults to 2 if not specified.

Using the Function

```
result1 = multiply(5)    # Uses default value for b
result2 = multiply(5, 3) # Uses specified value for b


print(result1)  # Output: 10 (5 * 2)
print(result2)  # Output: 15 (5 * 3)
```

Summary of Key Points

- Functions are defined using def.
- You can pass parameters to functions.
- Functions can return values using the return statement.
- You can create functions without parameters, with default parameters, or with multiple parameters.

Practice Questions

1. **Basic Addition Function**: Write a function named add_numbers that takes two numbers as parameters and returns their sum.
2. **Find Maximum**: Write a function named find_max that takes 4 user enter numbers and returns the largest number in the list.
3. **Check Prime**: Create a function named is_prime that checks if a number is prime and returns True or False.
4. **Sum of List**: Create a function called sum_list that takes a list of numbers and returns the sum of all the numbers in the list.
5. **Temperature Conversion**: Write a function named celsius_to_fahrenheit that converts a temperature from Celsius to Fahrenheit.

6Find minimu: write a fuction and find the minimum of 2 number.

Chapter 10

(File Handling In python)

File handling in Python is about reading from and writing to files on your computer. Here's a simple explanation of how it works:

Basics of File Handling in Python

1. **Opening a File**:
    - To work with a file, you first need to open it using the open() function.
    - Syntax:

file = open('filename.txt', 'mode')

    - **Modes**:
        - 'r': Read (default) - opens a file for reading.
        - 'w': Write - creates a new file or overwrites an existing file.
        - 'a': Append - adds to the end of an existing file.
        - 'b': Binary mode - used for binary files (like images).
2. **Reading from a File**:
    - Use methods like .read(), .readline(), or .readlines() to read the content.
    - Example:

content = file.read() # Reads the entire file

3. **Writing to a File**:

- Use the .write() method to add content to a file.
- Example:

```
file.write("Hello, World!")  # Writes to the file
```

4. **Closing the File**:
   - It’s important to close the file after you’re done to free up resources.
   - Use the .close() method.
   - Example:

```
file.close()  # Closes the file
```

5. **Using with Statement**:
   - Instead of manually closing the file, you can use a with statement, which automatically closes the file for you.
   - Example:

```
with open('filename.txt', 'r') as file:
    content = file.read()  # Automatically closes the file after this block
```

Example of File Handling

Here’s a simple example that shows how to write and read from a file:

```
# Writing to a file
with open('example.txt', 'w') as file:
    file.write("Hello, World!\n")
    file.write("This is a simple file handling example.")
```

```
# Reading from a file
with open('example.txt', 'r') as file:
    content = file.read()
    print(content)  # Prints the content of the file
```

Summary

- Use open() to access files.
- Use .read() and .write() to read from and write to files.
- Always close files with .close() or use with to handle it automatically

Practice

1. **Create a Text File**: Write a Python program that creates a text file named myfile.txt and writes "Hello, Python!" into it.
2. **Read a Text File**: Write a program that opens myfile.txt (from the previous question), reads its content, and prints it to the console.
3. **Append to a File**: Modify your program to append the text "This is a file handling exercise." to myfile.txt.
4. **Count Lines in a File**: Write a program that counts the number of lines in myfile.txt and prints the count.

Intermediate Level

5. **Copy File Content**: Write a program that copies the content of myfile.txt to a new file called copyfile.txt.
6. **Read and Print Specific Lines**: Write a program that reads myfile.txt and prints only the first two lines.
7. **Find** : Write a program that searches for a specific word in myfile.txt .
8. **Replace Text :** replaces(Q7) it with another word, saving the result in a new file called updatedfile.txt.
9. **Task:** Write a Python program that creates a file named greetings.txt and writes the following lline-hello dear welcome to here.

10. Tast: Write a Python program that creates a file named my_notes.txt, writes "This is my first note!" into it, and then reads the content of the file to print it.

Chapter 11

Oops and Inheritance

Object-Oriented Programming (OOP) in Python is a way to organize your code using "classes" and "objects." It makes your code simpler, more organized, and reusable.

Simple Explanation of OOP Concepts

1. **Classes**: Think of a class like a blueprint for making things. For example, a "Car" class is a blueprint for making different cars. It defines what properties (like color or brand) and actions (like drive or stop) a car can have.

Example:

```
class Car:
    def __init__(self, color, brand):
        self.color = color
        self.brand = brand


    def drive(self):
        print(f"The {self.color} {self.brand} is driving!")
```

2. **Objects**: Objects are the actual items created from the class blueprint. If a class is a blueprint for a car, then an object is the actual car itself.

Example:

python

Copy code

```
my_car = Car("red", "Toyota")

my_car.drive()  # Output: The red Toyota is driving!
```

Inheritance in Simple Words

Inheritance is like a way to create a new class based on an existing one. It allows the new class to use the properties and methods of the old class and add its own.

Think of it like a family: a child inherits features from their parents but also has their own unique characteristics.

Example of Inheritance

Let's say you have a class called Animal and you want to create a Dog class. Instead of writing everything from scratch, you can make the Dog class inherit from the Animal class.

python

Copy code

```
# Parent class

class Animal:

    def speak(self):

        print("Some generic animal sound")


# Child class
```

```
class Dog(Animal):  # Dog inherits from Animal
    def speak(self):  # Overriding the parent method
        print("Woof!")


# Creating objects
generic_animal = Animal()
generic_animal.speak()  # Output: Some generic animal sound


dog = Dog()
dog.speak()  # Output: Woof!
```

In this example:

- The Dog class inherits everything from the Animal class.
- It also changes the behavior of the speak() method to match a dog's sound.

In Simple Words

- **Classes** are like blueprints for objects.
- **Objects** are things created using those blueprints.
- **Inheritance** lets you create a new class based on an existing one, so you don't have to start from scratch. It helps you reuse code and make small changes to fit your needs.

OOP and inheritance make your code more organized, reusable, and easier to manage.

Practice

1. Create a class called Laptop with attributes brand and model.

2. Write a method inside the Laptop class called specs() that prints the brand and model.
3. Create an object of the Laptop class with brand "Dell" and model "Inspiron 15".
4. Create a class called Vehicle and add a method called start_engine() that prints "Engine started."
5. Create a class called Car that inherits from the Vehicle class and overrides the start_engine() method to print "Car engine started."
6. Write a class called Person with private attributes __name and __age, and methods to set and get these attributes.
7. Create a base class called Animal with a method called speak() and a subclass Dog that overrides the speak() method.
8. Implement a class called Shape with a method area() returning 0, and subclass Square that calculates the area based on its side length.
9. Write a function called describe_sound() that takes an object and calls its speak() method, demonstrating polymorphism.
10. Create a class Bird with a method fly(), a class Fish with a method swim(), and a subclass Penguin that inherits from both.

Chapter 12

(Modules , date and time, Random library)

Modules

A **module** is a single file containing Python code. It can define functions, classes, and variables. You can create your own modules or use built-in ones.

Creating a Module

1. **Create a file named mymodule.py:**

```
# mymodule.py

def greet(name):

    return f"Hello, {name}!"
```

2. **Using the Module:**

You can import and use your module in another Python file.

```
# main.py

import mymodule


print(mymodule.greet("Alice"))  # Output: Hello, Al
```

DATE and TIME

you can work with dates and times using the datetime module, which provides classes for manipulating dates and times.

Getting Current Date and Time

1. Import the module:

from datetime import datetime

2. Get current date and time:

python

Copy code

```
now = datetime.now()
print(now)  # Example output: 2024-10-18 12:34:56.789123
```

Formatting Date and Time

You can format the date and time into a readable string:

```
formatted = now.strftime("%Y-%m-%d %H:%M:%S")
print(formatted)  # Example output: 2024-10-18 12:34:56
```

Common Format Codes

- %Y: Year (e.g., 2024)
- %m: Month (01-12)
- %d: Day (01-31)
- %H: Hour (00-23)
- %M: Minute (00-59)
- %S: Second (00-59)

Creating a Specific Date

You can also create a specific date:

from datetime import date

```
my_date = date(2024, 10, 18)
print(my_date)  # Output: 2024-10-18
```

Summary

- Use datetime.now() to get the current date and time.
- Use strftime() to format dates and times into strings.
- Use date(year, month, day) to create a specific date.

random Module Overview

Purpose: Generates random numbers and selects random items.

Key Functions

1. Import the Module:

```
import random
```

2. Generate Random Numbers:
    - Random Float (0.0 to 1.0):

```
random.random()
```

    - Random Integer (inclusive):

```
random.randint(1, 10)  # Random int between 1 and 10
```

3. Select Random Items:
    - From a List:

```
random.choice(['red', 'green', 'blue'])  # Randomly selects an item
```

- Shuffle a List:

```
colors = ['red', 'green', 'blue']

random.shuffle(colors)  # Shuffles the list in place
```

4. Setting a Seed:
    - Reproducible Results:

```
random.seed(10)  # Sets the seed for reproducibility
```

Example

```
import random


# Generate a random integer

print(random.randint(1, 100))


# Shuffle a list

items = ['apple', 'banana', 'cherry']

random.shuffle(items)

print(items)


# Choose a random item

print(random.choice(items))
```

Summary

- Use random for generating random numbers and selecting random items.
- Functions include random(), randint(), choice(), and shuffle().

exception Handling

What is it?
Exception handling helps manage errors that occur in your code so that your program doesn't crash.

Key Parts

1. **Try Block:**
   Put code that might cause an error here.

```
try:
    # Code that might fail
    result = 10 / 0  # This will cause an error
```

2. **Except Block:**
   Specify what to do if an error happens.

```
except ZeroDivisionError:
    print("Error: You can't divide by zero!")
```

Complete Example

Here’s a simple example:

```
try:
    number = int(input("Enter a number: "))  # User input
```

```
    result = 10 / number  # Might cause ZeroDivisionError
    print("Result:", result)
except ValueError:
    print("Error: That's not a valid number!")
except ZeroDivisionError:
    print("Error: You can't divide by zero!")
```

Summary

- **Use try** to test code that might fail.
- **Use except** to handle the error.

Practice

Modules Practice Questions

1. Creating a Module:
    - Create a module named math_operations.py that contains two functions: add(a, b) which returns the sum of a and b, and subtract(a, b) which returns the difference.
2. Using a Module:
    - In a separate file, import your math_operations module and use both functions to perform calculations with two numbers of your choice.

Date and Time Practice Questions

1. Current Date and Time:
    - Write a program that prints the current date and time in the format YYYY-MM-DD HH:MM:SS.
2. Specific Date:
    - Create a date object for your birthday (or any specific date) and print it in the format DD/MM/YYYY.
3. Date Formatting:
    - Get the current date and format it to display only the month and year (e.g., October 2024).

Random Library Practice Questions

1. Random Integer:
    - Write a program that generates and prints a random integer between 1 and 50.
2. Random Choice from List:

- Create a list of five different fruits and write a program that randomly selects and prints one fruit from the list.

3. Shuffle a List:

    - Create a list of your favorite colors and write a program that shuffles the list and prints the shuffled result.